# CHAPTER 6

# SEMANTIC GRAPH MATCHING FOR MULTIMODAL REASONING WITH MGRAG

This chapter is based on *MGRAG: Semantic Subgraph Matching and Graph-Aware Caching for Multimodal Retrieval-Augmented Generation*, published in the *Proceedings of the VLDB Endowment*, volume 19, number 9, pages 2480–2493, 2026 [1]. It studies how a query can determine the evidence graph to construct, how paths can express semantic correspondences across different graph topologies, and how the resulting structure can guide context computation.

## 6.1 Multimodal Evidence and Query Requirements

An answer to a multimodal query may depend on a relation that is distributed across text and images. A document can name a person, an image can show an action, and another passage can connect that person to an event. Retrieving each item by its similarity to the query supplies candidate evidence, but the relation needed for the answer can remain implicit. A graph makes the entities and their connections available as a retrieval object. Its usefulness depends on which facts are extracted and on whether its representation can express the relation requested by the query.

This chapter considers a corpus $\mathscr{D} = \mathscr{D}_{\text{txt}} \cup \mathscr{D}_{\text{img}}$ and a query $q$ that can contain text, an image, or both. A retrieval module selects evidence from the corpus, and a multimodal language model generates an answer using that evidence. The research question is how to construct and retrieve a query-relevant relational representation while controlling the cost of preparing and processing the context. The answer-generation task remains grounded in the original documents and images. The graph supplies an additional representation of their query-related content.

Three decisions govern this process. The first concerns when to construct a graph. A corpus-wide graph requires extraction before the query is known. Its entities and relations consequently reflect the extraction prompt's general description of the corpus. When an image contains several people, objects, and actions, the details relevant to one query can differ from those relevant to another. MGRAG places the query inside the extraction process and constructs the graph incrementally from ranked evidence. The resulting graph is specific to the current information

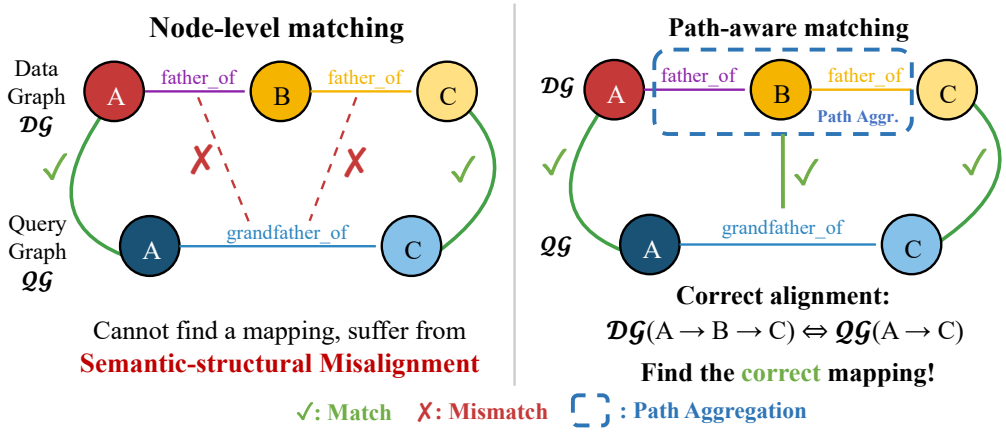



Figure 6.1: Path-level correspondence in the MGRAG motivating example. A `grandfather_of` relation corresponds to two `father_of` edges. Original Figure 1, p. 2481 [1].

requirement.

The second decision concerns what constitutes a match. A query and its supporting documents need not use the same decomposition of a fact. The published motivating example describes a data graph containing two `father_of` edges and a query containing one `grandfather_of` edge. The requested relation is represented by a two-hop path in the data. Requiring the same edge topology would exclude this correspondence. Figure 6.1 reproduces the example. MGRAG evaluates semantic representations of paths so that a query relation can be supported by a sequence of evidence relations.

The third decision concerns the cost of processing retrieved evidence. Graph construction already sends evidence to a language model to extract entities and relations. Some of that content appears again when the matched graph and its source chunks are used for answering. Reusing the earlier key-value (KV) states can reduce repeated computation. Their suitability depends on the new query and context, so MGRAG selectively recomputes token states. It combines attention scores with topological features of the facts associated with those tokens. Structure thus enters both evidence selection and the subsequent use of the selected context.

These decisions extend the thesis from connected textual evidence to multimodal evidence whose semantic decomposition is query-dependent. MGRAG develops its own graph construction, path-matching procedure, and cache score. The connection to the preceding studies lies in the role of explicit structure in organizing evidence. The evaluation in this chapter measures multimodal answer quality and context-serving costs under the published MGRAG protocol.

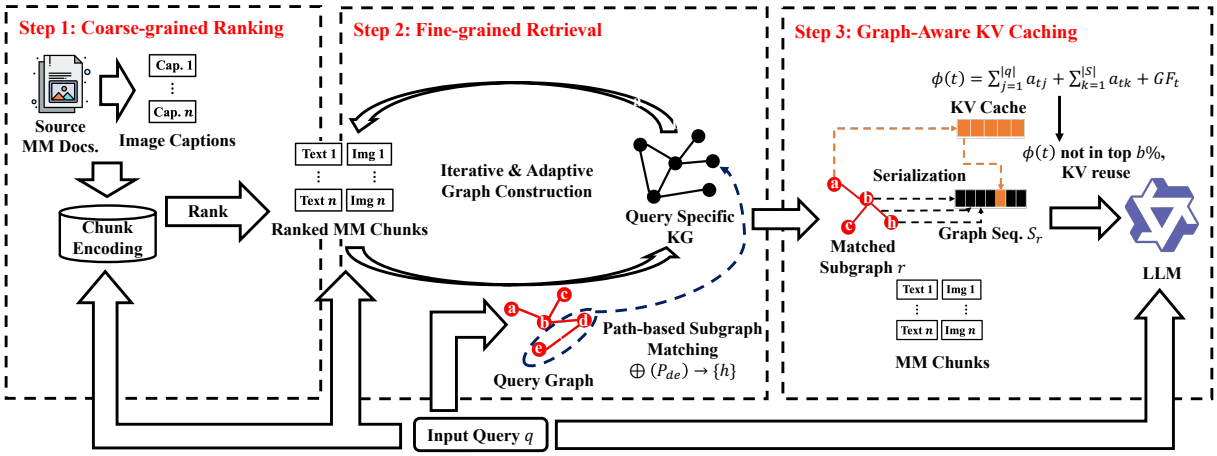



Figure 6.2: The MGRAG pipeline. Ranked chunks support incremental query-aware graph construction and path-based matching. The matched graph also supplies features for selective KV recomputation. Original Figure 2, p. 2484 [1].

## 6.2 Framework and Evidence Representations

Figure 6.2 shows three stages. Coarse-grained ranking orders text chunks and images according to their relevance to the query. Fine-grained retrieval consumes this ranked list, extracts query-related facts, and searches the growing graph for a semantic match. Graph-aware KV recomputation then determines which cached token states to update for answer generation. The stopping decision in the middle stage connects ranking, graph construction, and matching. Each additional chunk can change whether the graph contains a suitable correspondence.

Two graphs serve different purposes. The query graph $\mathcal{QG}_q = (\widehat{\mathcal{V}_q}, \widehat{\mathcal{E}_q})$ describes the entities and relations requested by $q$, including placeholders for unknown information. The evidence graph $\mathcal{KG}_q = (\mathcal{V}_q, \mathcal{E}_q)$ contains facts extracted from the chunks considered for that query. A mapping connects query entities to evidence entities. Paths between the mapped endpoints supply candidate evidence for the query relations.

For generation, MGRAG uses the graph sequence $S_r$, the original chunks $\mathcal{C}_q$ consumed during construction, and the query $q$. Keeping the chunks preserves access to text and visual evidence beyond the extracted triples. Their identity also links the selected graph to the material originally ranked and processed during construction. The graph gives the generator an explicit selection of relational content, while the source chunks provide the material from which the selection was made. The final multimodal context retains the original images alongside their extracted relational representation.

## 6.3 Coarse-grained Ranking

MGRAG first divides textual documents into overlapping chunks. The published setting uses a chunk length $l_c = 256$ tokens and overlap $l_o = 32$ tokens. Images form the other members of the chunk collection, giving $\mathscr{C} = \mathscr{T} \cup \mathscr{D}_{\text{img}}$, where $\mathscr{T}$ denotes the text chunks. This collection is the input to the coarse retrieval stage. Ranking identifies the order in which content will be examined, rather than directly defining the final graph context.

The implementation uses the text encoder bge-base-en-v1.5 [2]. Text chunks are encoded directly. An image is represented for ranking by its caption, which can be supplied by the dataset or generated by a language model. Both types of evidence can therefore be compared in the text embedding space. Let $\mathscr{M}$ denote that encoder and $\mathbf{h}_c$ the representation of a candidate chunk. For an image chunk, the input to $\mathscr{M}$ is its caption. This detail matters because the ranking representation expresses the image through language, whereas the later extraction and answering stages can use its visual content.

For a query containing text $q_{\text{txt}}$ and an image caption $q_{\text{cap}}$, MGRAG averages the two representations,

$$\mathbf{h}_q = \frac{\mathscr{M}(q_{\text{txt}}) + \mathscr{M}(q_{\text{cap}})}{2}. \tag{6.1}$$

A query with one modality uses its corresponding textual representation. The distance to a candidate is

$$d(q, c) = \|\mathbf{h}_q - \mathbf{h}_c\|_2, \tag{6.2}$$

and the chunks are ordered by increasing distance. Equations 6.1 and 6.2 retain the averaging and Euclidean-ranking operations of the published method.

This stage gives a query-dependent priority to relatively inexpensive chunk representations. Fine-grained processing begins with the nearest chunk and can move down the list when its current graph is insufficient. A high-ranked chunk is consequently a candidate for relational extraction; it is not automatically accepted as sufficient evidence. Conversely, graph matching can stop construction before every candidate has been processed. The coarse-to-fine design separates the cost of ordering the corpus from the cost of extracting its query-related facts.

Caption quality affects which images receive an early position. A caption can omit a visual detail that becomes relevant during graph construction, or foreground content unrelated to the query. The published CLIP substitution experiment tests an alternative coarse representation within this pipeline. Its results are discussed alongside the component ablations in Section 6.11, where the encoder choice can be assessed under the same answering task.

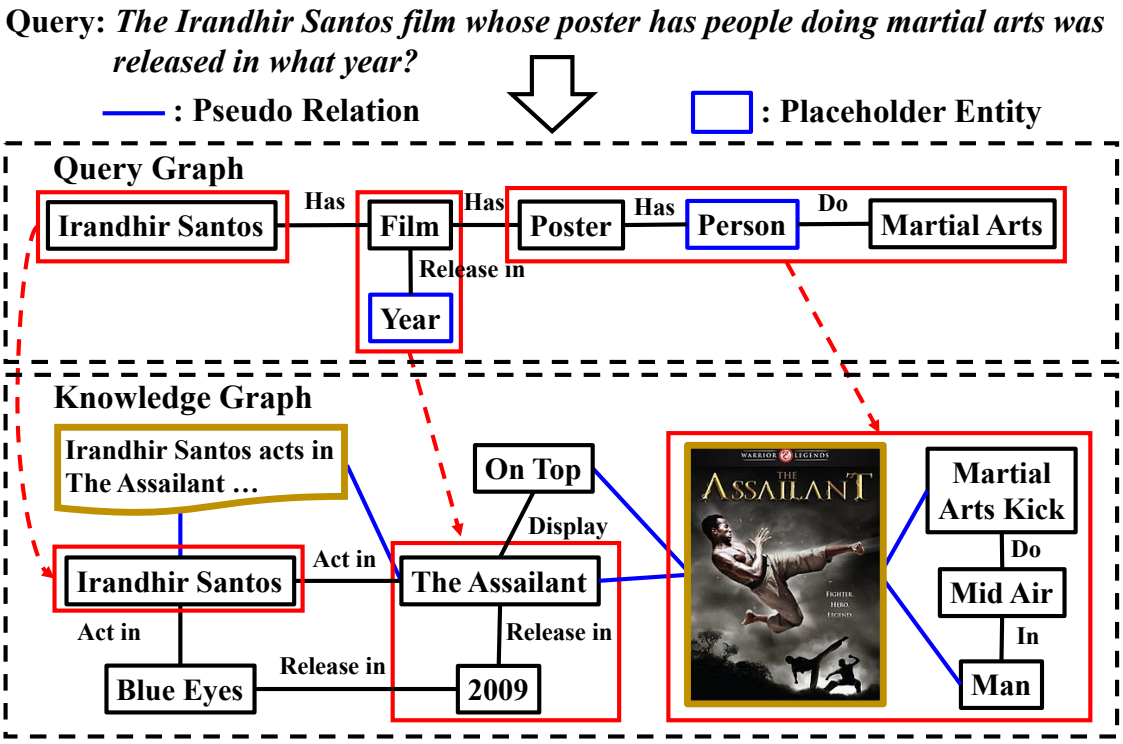



Figure 6.3: Query and evidence graphs in path-based semantic matching. Placeholder entities express unknown query information; pseudo-relations connect passage nodes to extracted entities. Original Figure 3, p. 2485 [1].

## 6.4 Query-aware Graph Construction

### 6.4.1 Representing the query

MGRAG asks the language model to extract entities and relations from the query. Some query entities are known, such as the name of a person or an object appearing in the question. Others are requested by the question. The extraction prompt represents those unknown components using semantically meaningful placeholders, including types such as *Person* or *Year*. A placeholder provides a textual representation that can be compared with evidence entities even though its value has not yet been supplied.

Figure 6.3 shows the query and evidence graph representations. The placeholder boxes describe missing knowledge in the query. Passage nodes connect extracted entities back to their chunks. These connections let the graph carry both relational content and its association with the original multimodal evidence. In a multimodal corpus, a passage node can represent an image as well as a text chunk.

The original example asks which year an Irandhir Santos film was released when its poster shows people doing martial arts. The query graph connects the actor to an unknown film, the film

to a poster, and a person on the poster to an action. It also includes a release-year requirement. The evidence graph contains named film candidates and a visual description of a man performing a martial-arts kick in midair. The depicted film, *The Assailant*, is connected to the year 2009. These facts are the content of the published illustrative example.

The example separates identifying the film from reading its release year. The actor's name alone can lead to several film candidates, including the other film shown in the graph. The visual relation supplies the condition that distinguishes the requested candidate. Once the candidate is connected to the poster evidence, the release relation provides the requested value. The query therefore needs a connected account of the actor, film, depicted action, and year. A list containing a similar actor passage and an unrelated year passage would leave that correspondence unresolved.

The placeholders give the matcher explicit locations for these missing values. The film placeholder can be compared with named film entities, while the year placeholder marks the relation whose endpoint supplies the requested answer. Representing unknown values by their semantic roles lets matching operate before those values are known. The path procedure then tests whether the selected evidence entities are connected through semantically suitable relations. This is the point at which the query's information requirement becomes a constraint on the retrieved graph.

The query graph expresses a retrieval requirement at entity and relation granularity. Its construction is a model-mediated interpretation of the question. The downstream matcher operates on this interpretation, including the wording chosen for placeholders and relations. A semantic match therefore indicates agreement between the represented query and represented evidence. Answer quality is evaluated separately using the original question and reference answer.

### 6.4.2 Extracting facts from ranked chunks

For a chunk $c$, the extraction model receives the chunk, query, and extraction prompt $p_{\text{ext}}$. Its output contains query-related entities and relations,

$$(\mathscr{V}_q^c, \mathscr{E}_q^c) = \text{LLM}(c, q, p_{\text{ext}}). \tag{6.3}$$

Including $q$ makes the extraction selective. The model is asked to identify events and relations relevant to the current question, rather than to enumerate every describable fact in the chunk. For an image, this permits attention to query-related visual events; for text, it permits selection of the entities and relationships needed to describe the requested evidence.

The chunk is also added as a passage node. MGRAG connects this node to the entities

extracted from it using the pseudo-relation `consists_in`, retaining the published relation name and orientation from chunk to entity. If $\mathscr{C}_q^{(i)}$ is the prefix of ranked chunks consumed after iteration $i$, the cumulative graph contains

$$\mathscr{V}_q^{(i)} = \bigcup_{c \in \mathscr{C}_q^{(i)}} (\mathscr{V}_q^c \cup \{c\}), \tag{6.4}$$

$$\mathscr{E}_q^{(i)} = \bigcup_{c \in \mathscr{C}_q^{(i)}} \left(\mathscr{E}_q^c \cup \{(c, \texttt{consists_in}, v) : v \in \mathscr{V}_q^c\}\right). \tag{6.5}$$

The iteration-indexed notation makes explicit that the unions range over the chunks already processed. It preserves the incremental operation described in the published construction procedure.

Passage nodes give the matcher a route between a fact and its context. They can also connect facts associated with the same source item. The resulting graph combines extracted semantic relations with document-membership relations. These two edge types have different interpretations, even though both participate in the evidence representation. The distinction is visible in the original example's use of separate edge colors.

### 6.4.3 Matching as a stopping decision

After each graph update, MGRAG attempts path-based semantic subgraph matching against the query graph. A valid match ends the construction loop. If the matcher returns an empty result, the next ranked chunk is used to enrich the graph. The loop is bounded by the maximum number of construction iterations $\mu$. Thus, the amount of graph construction depends on both the ranking order and the matching outcome.

This design uses semantic coverage as an operational stopping test. Coverage is evaluated through the query graph's entities and relations, so a match requires more than one highly similar isolated entity. The procedure seeks a correspondence for the query's relational requirements. When the current graph lacks such a correspondence, additional evidence can introduce a missing endpoint, a relation, or a path between existing endpoints. The graph can therefore become matchable through information accumulated across chunks.

The iteration cap controls the amount of evidence examined. It also bounds the opportunity to recover a fact that occurs late in the ranking. Reaching the cap and finding a valid match are distinct termination events. The matcher returns an empty result when no valid correspondence exists in its current graph. The cap, graph extraction, and matching threshold jointly determine which context reaches the answer generator.

## 6.5 Path-based Semantic Subgraph Matching

### 6.5.1 From edges to paths

A relation in the query graph can be expressed by several relations in the evidence graph. MGRAG assigns text embeddings to entity and relation descriptions. For a path $P(u, v)$ containing relations $r_1, \dots, r_k$, it defines a semantic representation by aggregating the relation embeddings,

$$\mathbf{h}_{P(u,v)} = \bigoplus_{j=1}^{k} \mathbf{h}_{r_j}. \tag{6.6}$$

The aggregation operator can use mean pooling. This operation provides a representation of the composed evidence relation that can be compared with a query relation. The published greedy expansion further evaluates a joint representation containing the current path, candidate edge, and candidate endpoint. These are the path representation and extension test used at different steps of the procedure.

Let $I : \widehat{\mathscr{V}_q} \to \mathscr{V}_q$ be a candidate entity mapping. For a query edge $(u, \hat{r}, v)$, the candidate evidence paths run from $I(u)$ to $I(v)$. Denote this collection by $\mathscr{P}(I(u), I(v))$. The matching objective is

$$\max_{I} \sum_{(u,\hat{r},v)\in\widehat{\mathscr{E}_q}} \max_{P\in\mathscr{P}(I(u),I(v))} \cos\big(\mathbf{h}_{\hat{r}}, \mathbf{h}_P\big). \tag{6.7}$$

Candidate mappings must satisfy the semantic threshold $\theta$ for the mapped entities and for every query relation's supporting path,

$$\cos\big(\mathbf{h}_u, \mathbf{h}_{I(u)}\big) > \theta \qquad \text{for all } u \in \widehat{\mathscr{V}_q}, \tag{6.8}$$

$$\max_{P\in\mathscr{P}(I(u),I(v))} \cos\big(\mathbf{h}_{\hat{r}}, \mathbf{h}_P\big) > \theta \qquad \text{for all } (u, \hat{r}, v) \in \widehat{\mathscr{E}_q}. \tag{6.9}$$

The objective and constraints follow the published formulation. Their unit of semantic validation is a query edge and an associated evidence path. The selected evidence is the subgraph formed from the mapped entities and accepted paths, which can include intermediate entities absent from the query graph.

For a fixed candidate mapping, the inner maximization in Equation 6.7 selects a supporting path for each query relation. The outer search compares entity mappings after those relation scores have been obtained. These levels distinguish the problem of locating the query's entities from the problem of explaining their relationships. Two mappings can have similar endpoint em-

beddings yet differ in whether their connecting paths satisfy the requested relation. Conversely, several paths between the same mapped endpoints can express different relations and receive different semantic scores.

The returned subgraph is assembled from the paths needed by the accepted query edges. Paths can share intermediate entities and edges. Their union exposes those shared facts in the evidence graph, while the query graph continues to specify which relational requirements were validated. A graph assembled in this way need not have the same number of entities as the query graph. Intermediate evidence nodes are precisely what permits a relation expressed compactly in the question to be supported by a more detailed description in the corpus.

This representation relaxes the requirement that the two graphs express a fact with the same number of edges. In the family-relation example, the evidence path has two steps and the query relation has one. Their correspondence is judged through semantic similarity. The procedure retains endpoint requirements, so the selected relation must connect entities that themselves correspond to the query. It combines entity-level alignment with path-level evidence instead of treating either component as sufficient on its own.

### 6.5.2 Anchor selection and candidate mappings

MGRAG first computes pairwise cosine similarities between query entities and evidence entities. The highest-similarity pair supplies an anchor entity in the query graph and its candidate counterpart in the evidence graph. If no valid match is obtained from that anchor, the method tries other anchors. An anchor limits the subsequent search to mappings consistent with its correspondence.

Candidate mappings are enumerated using a similarity-based modification of VF2 [3]. VF2 supplies a graph-matching search procedure; the MGRAG adaptation uses the threshold $\theta$ in semantic feasibility checks. Each candidate is then validated relation by relation through greedy path search. A candidate entity correspondence can consequently be rejected because one of its required relations lacks an acceptable evidence path.

For each accepted mapping, the algorithm takes the union of the paths that support its query edges. It averages the edge-wise semantic similarities to obtain a score $s$. The highest-scoring accepted subgraph is returned. Dividing by the number of query edges does not change the ranking of candidate mappings for the same query. It provides an average semantic score with a common range across queries whose graph sizes differ.

**Algorithm 1** Path-based semantic matching in MGRAG

**Require:** Query graph $\mathcal{QG}_q$, evidence graph $\mathcal{KG}_q$, threshold $\theta$

Initialize valid results $\mathcal{R} \leftarrow \varnothing$
Select an anchor correspondence and enumerate candidate mappings
**for** each candidate mapping $I$ **do**
  Initialize the subgraph with mapped query entities; set $s \leftarrow 0$
  **for** each query edge $(u, \hat{r}, v)$ **do**
    Search greedily for a semantically valid path from $I(u)$ to $I(v)$
    **if** the path search fails **then**
      Reject $I$ and stop validating this mapping
    **else**
      Add the path's nodes and edges to the candidate subgraph
      Add its cosine similarity to $s$
    **end if**
  **end for**
  **if** every query edge has a valid path **then**
    Add the subgraph and score $s/|\widehat{\mathcal{E}}_q|$ to $\mathcal{R}$
  **end if**
**end for**
Return the highest-scoring result, or $\varnothing$ if $\mathcal{R}$ is empty

### 6.5.3 Greedy semantic path growth

The path search begins at the mapped source entity and maintains its current endpoint $x$. For each neighboring entity $y$ and connecting edge $(x, r, y)$, MGRAG evaluates the candidate extension using

$$\cos\left(\mathbf{h}_P \oplus \mathbf{h}_{e(x,r,y)} \oplus \mathbf{h}_y, \mathbf{h}_{\hat{e}(u,\hat{r},v)}\right). \tag{6.10}$$

Among extensions exceeding $\theta$, it selects the highest-scoring one, appends that edge to the path, and updates the endpoint to $y$. This repeats until the path reaches the mapped target. The search fails when no qualifying extension is available. The experiments also impose a maximum path length $l_p$ to bound expansion.

The endpoint and threshold checks serve different roles. Reaching the target gives the evidence path the required relational endpoints. Passing the semantic checks gives it a representation consistent with the requested relation. An attractive local extension can still prevent the greedy search from reaching the target, so a failed search does not establish that all possible paths fail. The algorithm returns the best result among the valid candidates it finds.

All query edges must pass path validation before a mapping is accepted. This condition prevents a single high-scoring relation from compensating for an unsupported required relation. The selected subgraph may contain paths of different lengths, because the evidence required

by different query edges can have different decompositions. Construction then stops on that accepted match; otherwise, a further ranked chunk is incorporated and matching is attempted again.

### 6.5.4 Published algorithmic properties

The publication states that path-based semantic subgraph matching is NP-hard. Its proof sketch reduces subgraph isomorphism to a restricted instance with uniform embeddings and unit path lengths. The result motivates the use of semantic path growth and constrained mapping enumeration. The complexity of enumerating mappings remains relevant even when each individual extension is selected greedily.

The published Theorem 2 gives a threshold-based quality condition for a valid returned match. Every matched query relation has a supporting path with cosine similarity at least $\theta$. Under the strict acceptance test in Equation 6.9, the average score of an accepted mapping exceeds $\theta$ as well. This condition describes the semantic validation of returned evidence. It is conditional on finding a valid match and does not require the algorithm to recover one whenever such a subgraph exists.

The technical report discusses the threshold in terms of a bottleneck transport cost [4]. With a per-edge cost of one minus cosine similarity, the threshold limits the largest accepted local cost. The explicit theorem statement in the publisher PDF is used here to specify the result. The method's empirical quality is assessed by the graph-matching substitutions and answer-quality measurements in Section 6.9.

### 6.5.5 Serializing selected evidence

After matching, MGRAG serializes the selected triples into $S_r$, separating facts with `[SEP]`. This sequence is combined with $q$ and the original chunks consumed during construction. The serialization supplies an explicit list of entities and relationships to the answering model. Original chunks remain available because extraction is a selective description of the evidence, and some linguistic or visual details may lie outside the triples.

The same entities and relations were involved in the earlier graph-construction calls. Their tokens can therefore overlap with tokens in the answering context. MGRAG uses this relationship to reuse cached computation from construction while updating states important to the new answering context. The graph produced during retrieval thus becomes an input to the context-computation policy as well as to the language model's prompt.

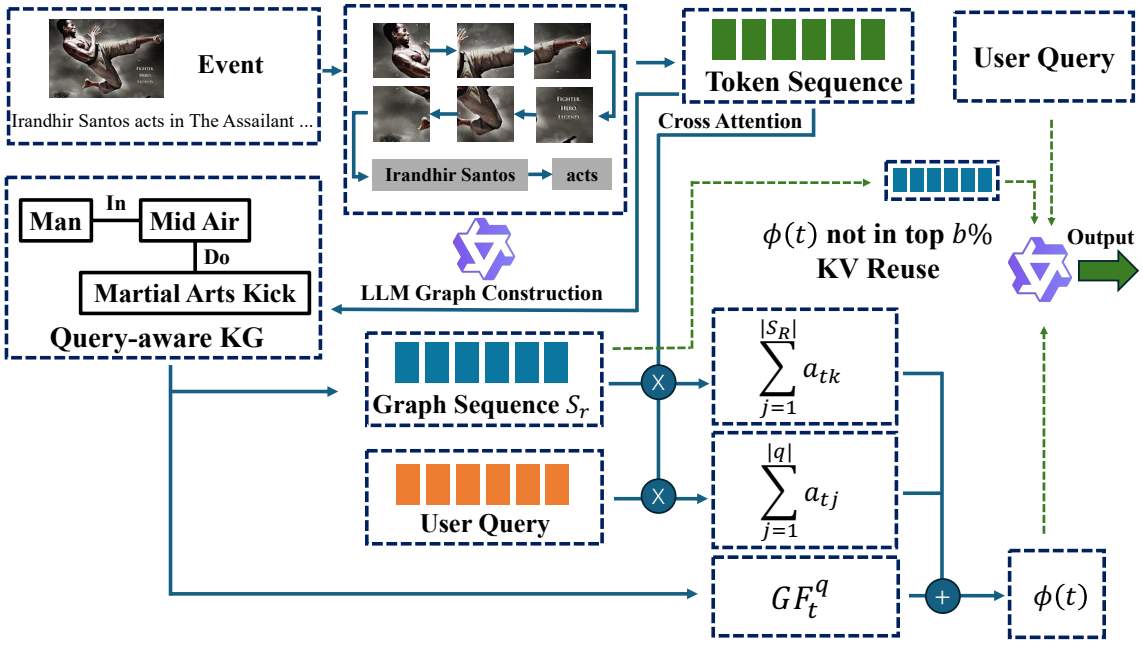



Figure 6.4: Graph features supplement attention in token recomputation scoring. The diagram retains the original graphical content. Original Figure 4, p. 2487 [1].

## 6.6 Graph-aware KV Recomputation

### 6.6.1 Reuse and selective updates

A transformer caches key and value projections so that earlier token states can be reused. In MGRAG, content processed during graph construction appears again in the query-answering context. Recomputing every state repeats part of this work. Reusing every state can retain representations calculated under a different surrounding sequence. The cache policy chooses a subset for recomputation and reuses the others.

Let $\phi(t)$ be the recomputation score of token $t$. At each transformer layer, MGRAG recomputes the tokens in the highest $b\%$ of scores and reuses the cached states for the remaining tokens. The published description uses a fixed-size cache with least-recently-used eviction. Its proposed policy concerns the ranking of token states for selective recomputation. The budget is represented by $b$; the numerical experiment tables do not specify a fixed value for that budget.

Figure 6.4 shows how the graph contributes. Query-related events are represented by entities and relations. The score combines attention to the query, attention to the graph sequence, and a feature derived from the topology of the associated fact. The goal of the added term is to give tokens access to an event-level structural signal alongside token-level attention.

### 6.6.2 Topological features of facts

MGRAG uses degree, betweenness, eigenvector, and closeness centrality to characterize a node. Degree centrality measures its normalized number of adjacent connections. Betweenness characterizes its participation in paths connecting other nodes. Eigenvector centrality weights connections by the centrality of their neighbors. Closeness uses distances to other graph nodes. The published method averages these four quantities,

$$f(u) = \frac{dc(u) + bc(u) + ec(u) + cc(u)}{4}. \tag{6.11}$$

For a token associated with a triple $(u, r, v)$, its graph feature is the mean of the endpoint scores,

$$GF_t^q = \frac{f(u) + f(v)}{2}. \tag{6.12}$$

The token-to-fact association follows the entity and relation token spans in the serialized representation. Tokens referring to the same fact therefore receive a shared structural component.

The graph feature concerns an entity's position in the query-specific graph. A node that connects several extracted facts can carry a different score from an isolated endpoint. Because extraction is query-dependent, the feature is calculated from a representation already selected for that query. Its meaning is relational importance within that graph, rather than a corpus-wide importance measure.

The published graphs are small, with fewer than twenty entities on average according to the report. Averaging four graph statistics provides a direct structural feature for these graphs. The method does not train a separate token-importance predictor on the evaluation queries. Its score instead uses the graph and attention information available during processing.

### 6.6.3 Combined recomputation score

Let $a_{tj}$ denote attention between token $t$ and token $j$, and let $|q|$ and $|S_r|$ denote the query and serialized-graph lengths. The score is

$$\phi(t) = \sum_{j=1}^{|q|} a_{tj} + \sum_{k=1}^{|S_r|} a_{tk} + GF_t^q. \tag{6.13}$$

The first term expresses interaction with the current question. The second expresses interaction with the selected relational context. The third assigns the structural feature of the associated

graph fact. The resulting sum determines which token states consume the recomputation budget.

The publication compares the policy with vLLM prefix caching [5] and CacheBlend [6]. Their retrieval pipelines and cache variants are identified separately in the evaluation. The inverse ablations also evaluate graph-aware recomputation within an existing retrieval pipeline. These comparisons expose answer-quality and time changes under several compositions of retrieval and caching.

## 6.7 Computational Cost

The published offline encoding cost is $O(|\mathscr{C}|d_{\text{emb}}l_c^2)$, with embedding dimension $d_{\text{emb}}$. Using $k$ processed chunks, query-graph vertex and edge sets $\widehat{\mathscr{V}}$ and $\widehat{\mathscr{E}}$, evidence-graph sets $\mathscr{V}$ and $\mathscr{E}$, maximum path length $l_p$, and language-model hidden dimension $d_{\text{LLM}}$, its reported online bound is

$$\begin{aligned} O\big(&d_{\text{emb}}kl_c^2 + l_p|\widehat{\mathscr{E}}|\,|\mathscr{V}|^{|\widehat{\mathscr{V}}|+1} + |\mathscr{V}||\mathscr{E}| \\ &+ d_{\text{LLM}}(|S_r|^2 + |q|^2)\big). \end{aligned} \tag{6.14}$$

The mapping-enumeration term is exponential in query-graph size. Query-specific construction and a bounded path length restrict the graphs processed in the evaluated setting. Actual module costs provide the complementary evidence about where runtime is spent. In particular, graph construction calls the language model repeatedly, whereas matching operates on the extracted graph. Section 6.10 reports these costs individually.

## 6.8 Experimental Protocol

The evaluation uses MMQA [7] and the Arxiv, Manual, Web, Wiki, Wit, and Recipe subsets of MRAMG-Bench [8]. Table 6.1 preserves the published dataset statistics. MMQA includes textual and image queries. The six MRAMG subsets contain textual queries over multimodal corpora. Their query-hop ranges and corpus sizes differ, so results are reported by dataset.

Table 6.1: Evaluation datasets, reproduced from published Table 3, p. 2488 [1]. T and I denote textual and image query modalities. The columns comprise MMQA and six MRAMG subsets.

| Statistic | MMQA | Wiki | Wit | Recipe | Arxiv | Manual | Web |
|---|---|---|---|---|---|---|---|
| Query Modality | T / I | T | T | T | T | T | T |
| Queries | 951 | 500 | 600 | 2,360 | 200 | 390 | 750 |

Table 6.1 (continued)

| Statistic | MMQA | Wiki | Wit | Recipe | Arxiv | Manual | Web |
|---|---|---|---|---|---|---|---|
| Texts | 218,285 | 538 | 639 | 1,528 | 101 | 40 | 1,500 |
| Images | 57,058 | 538 | 639 | 8,569 | 337 | 2,607 | 1,500 |
| Corpus Size | 22.5M | 680k | 478k | 1.1M | 136k | 420k | 215k |
| Query Hops | 1–2 | 2–5 | 2–4 | 1–6 | 3–4 | 2–6 | 2–3 |
| Avg Query Hops | 1.5 | 3.0 | 2.8 | 3.3 | 3.1 | 3.7 | 2.4 |

The shared generation backbone is Qwen2.5-VL-7B-Instruct [9]. The experiments run on an Intel Xeon Gold 5220R CPU at 2.20 GHz and one NVIDIA A100-SXM4 GPU with 40 GB memory. The retrieval amount is set to five, text chunks use 256 tokens, and overlap is 32 tokens. HippoRAG2 uses Contriever for dense passage retrieval [10]; the other reported embedding configurations use bge-base-en-v1.5. Similarity-based retrieval represents images through their captions. These are the published controls for the comparisons in this chapter.

VanillaRAG is implemented with LangChain and FAISS [11]. The GraphRAG implementation is nano-graphrag [12], and the remaining baseline systems use their released implementations. The protocol selects hybrid search for LightRAG and local search for GraphRAG. vLLM accelerates the tested baseline systems, while MGRAG uses its graph-aware cache procedure. Other baseline hyperparameters follow the implementations' defaults.

Four metrics characterize answers. Semantic similarity (SIM.) is the cosine similarity between bge-base-en-v1.5 representations of generated and reference answers. Accuracy (ACC.) is the reported proportion of correct answers. ROUGE-L (ROG.) evaluates longest-common-subsequence overlap [13], and BLEU-1 evaluates modified unigram precision [14]. The paper uses the RAGAS evaluation framework [15]. The tables preserve its metric names and precision, including the label BLEU used for BLEU-1.

The four answer metrics reflect different properties. Embedding similarity compares semantic proximity at the answer level. ROUGE-L and BLEU-1 retain sensitivity to the wording shared with the reference. Accuracy provides the task's correctness measurement. They are shown together because agreement in one measure does not imply identical behavior in the others. The publication reports point estimates for these experiments.

The complete-system baselines use different retrieval units. VanillaRAG returns dense-retrieved chunks, and R1-Searcher incorporates reinforcement learning into search and question answering. GraphRAG organizes its graph into communities and prepares summaries; LightRAG retrieves graph elements and neighborhoods through its selected search mode. Hip-

poRAG2 adds passage nodes and uses Personalized PageRank for retrieval. In the published experiment, these systems are compared as implemented pipelines. Their results include their own indexing and evidence-selection choices, while the shared backbone and evaluation metrics provide the common answering conditions.

The construction and matching substitutions supply a second level of comparison. Their role is to replace one part of MGRAG while retaining the surrounding question-answering task. For example, the graph-construction alternatives change which facts are available before matching. A matching alternative changes how the existing representation is aligned with the query. This separation is useful when an answer contains information from several source items: a missing fact and an unsuitable correspondence can both lower answer quality, but they concern different stages of the pipeline.

The comparisons have four roles. Complete-system baselines measure the combined retrieval and generation pipeline. Graph-construction substitutions replace the evidence extraction component. Graph-matching substitutions replace the correspondence mechanism. Cache comparisons and inverse ablations vary context-serving components. Keeping these roles distinct makes it possible to relate each experiment to the particular decision it tests.

## 6.9 Answer Quality and Structural Retrieval

### 6.9.1 Complete-system comparisons

Table 6.2 reproduces the complete-system results. MGRAG has the highest reported value in each of the four metric rows for each dataset. On MMQA, its accuracy is 0.7442, compared with 0.7053 for VanillaRAG and 0.6335 for HippoRAG2. Its semantic similarity is 0.8533, ROUGE-L is 0.6763, and BLEU-1 is 0.7215. GraphRAG and LightRAG have missing MMQA entries because those experiments exceeded three days. The dash records the time limit and supplies no quality measurement for those systems.

Table 6.2: Answer quality with Qwen2.5-VL-7B-Instruct, reproduced from published Table 4, p. 2489 [1]. Vanilla., R1., Graph., Light., and Hippo. denote VanillaRAG, R1-Searcher, GraphRAG, LightRAG, and HippoRAG2. A dash means that the experiment exceeded three days; it is not a zero score. Bold and underlining retain the publication's highlighting.

| Dataset | Metric | Vanilla. | R1. | Graph. | Light. | Hippo. | MGRAG |
|---|---|---|---|---|---|---|---|
| MMQA | SIM. | 0.8362 | 0.5478 | - | - | <u>0.8381</u> | **0.8533** |
| MMQA | ACC. | <u>0.7053</u> | 0.3567 | - | - | 0.6335 | **0.7442** |

Table 6.2 (continued)

| Dataset | Metric | Vanilla. | R1. | Graph. | Light. | Hippo. | MGRAG |
|---|---|---|---|---|---|---|---|
| MMQA | ROG. | 0.5942 | 0.1041 | - | - | 0.6659 | **0.6763** |
| MMQA | BLEU | 0.6645 | 0.0817 | - | - | 0.5917 | **0.7215** |
| Arxiv | SIM. | 0.8664 | 0.7051 | 0.7943 | 0.7404 | 0.8293 | **0.8760** |
| Arxiv | ACC. | 0.6638 | 0.3775 | 0.2350 | 0.3813 | 0.4688 | **0.6800** |
| Arxiv | ROG. | 0.2713 | 0.1189 | 0.1553 | 0.1580 | 0.2331 | **0.2726** |
| Arxiv | BLEU | 0.2452 | 0.1092 | 0.1690 | 0.1370 | 0.1281 | **0.2498** |
| Manual | SIM. | 0.8166 | 0.6046 | 0.5453 | 0.7255 | 0.6852 | **0.8221** |
| Manual | ACC. | 0.5881 | 0.2071 | 0.1436 | 0.4096 | 0.2641 | **0.6173** |
| Manual | ROG. | 0.3509 | 0.1078 | 0.0974 | 0.1595 | 0.2047 | **0.3839** |
| Manual | BLEU | 0.2767 | 0.0740 | 0.0718 | 0.1385 | 0.1314 | **0.3093** |
| Web | SIM. | 0.6937 | 0.6620 | 0.5470 | 0.5513 | 0.5642 | **0.7009** |
| Web | ACC. | 0.4703 | 0.4200 | 0.1053 | 0.2000 | 0.3607 | **0.5123** |
| Web | ROG. | 0.1508 | 0.1142 | 0.1133 | 0.0905 | 0.0616 | **0.1664** |
| Web | BLEU | 0.0326 | 0.0785 | 0.1031 | 0.0206 | 0.0133 | **0.1456** |
| Wiki | SIM. | 0.8824 | 0.5478 | 0.6966 | 0.7517 | 0.8204 | **0.8838** |
| Wiki | ACC. | 0.7800 | 0.3567 | 0.2000 | 0.3660 | 0.6500 | **0.7845** |
| Wiki | ROG. | 0.3559 | 0.1418 | 0.1361 | 0.2008 | 0.3042 | **0.3579** |
| Wiki | BLEU | 0.3053 | 0.1146 | 0.1007 | 0.1863 | 0.1851 | **0.3174** |
| Wit | SIM. | 0.7851 | 0.6110 | 0.6211 | 0.6973 | 0.7230 | **0.8033** |
| Wit | ACC. | 0.6675 | 0.6017 | 0.1500 | 0.2483 | 0.5138 | **0.6967** |
| Wit | ROG. | 0.3255 | 0.1307 | 0.1319 | 0.2416 | 0.2712 | **0.3589** |
| Wit | BLEU | 0.2095 | 0.0934 | 0.1142 | 0.1928 | 0.1200 | **0.2382** |
| Recipe | SIM. | 0.8599 | 0.6155 | 0.8166 | 0.8118 | 0.7792 | **0.8673** |
| Recipe | ACC. | 0.6525 | 0.2180 | 0.2771 | 0.3334 | 0.4917 | **0.6814** |
| Recipe | ROG. | 0.3714 | 0.1065 | 0.1745 | 0.2128 | 0.3303 | **0.4220** |
| Recipe | BLEU | 0.2599 | 0.0773 | 0.1527 | 0.1648 | 0.1941 | **0.3102** |

The advantage over the strongest baseline varies by dataset and metric. On Wiki, accuracy is 0.7845 for MGRAG and 0.7800 for VanillaRAG, while their semantic similarities are 0.8838 and 0.8824. On Web, the corresponding accuracies are 0.5123 and 0.4703. On Recipe, MGRAG reports ROUGE-L of 0.4220 compared with 0.3714 for VanillaRAG. The reported gains vary with the dataset and metric.

Several graph-based baselines score below VanillaRAG in this multimodal protocol. For

example, on Manual, GraphRAG, LightRAG, and HippoRAG2 report accuracies of 0.1436, 0.4096, and 0.2641, compared with VanillaRAG's 0.5881. The presence of a graph by itself therefore does not predict the ranking in this table. MGRAG's results concern its specific construction, matching, and context-processing design. The substitutions below help identify which parts contribute under the same evaluation framework.

### 6.9.2 Construction and matching substitutions

Table 6.3 reorganizes every value from the published graph-construction and matching comparison. Stanford OpenIE [16] and the statistical extraction alternative STAT replace the construction mechanism. VF2 and ISMAGS [3, 17] provide exact-matching alternatives. SM, VJ, RRWM, and IPFP represent approximate graph matching [18–21]. BGE, SLOTAlign, and FGWAlign supply semantic alternatives [2, 22, 23]. The table preserves the publication's three-decimal precision for this experiment.

Table 6.3: Graph-construction and graph-matching substitutions, transposed from published Table 7, p. 2490 [1]. All twelve method columns and all four metric values per dataset are retained. O.IE and STAT replace graph construction; the other alternatives replace matching. The shared backbone is Qwen2.5-VL-7B-Instruct. Original decimal precision and highlighting are preserved.

| Dataset | Method | SIM. | ACC. | ROG. | BLEU |
|---|---|---|---|---|---|
| MMQA | O.IE | 0.642 | 0.571 | 0.510 | 0.483 |
| MMQA | STAT | 0.655 | 0.602 | 0.519 | 0.500 |
| MMQA | VF2 | <u>0.851</u> | <u>0.735</u> | <u>0.673</u> | <u>0.620</u> |
| MMQA | ISMAGS | 0.833 | 0.706 | 0.638 | 0.590 |
| MMQA | SM | 0.832 | 0.702 | 0.632 | 0.582 |
| MMQA | VJ | 0.812 | 0.715 | 0.646 | 0.606 |
| MMQA | RRWM | 0.838 | 0.698 | 0.647 | 0.599 |
| MMQA | IPFP | 0.773 | 0.633 | 0.601 | 0.585 |
| MMQA | BGE | 0.832 | 0.703 | 0.635 | 0.586 |
| MMQA | SLOT | 0.838 | 0.700 | 0.650 | 0.602 |
| MMQA | FGW | 0.837 | 0.697 | 0.647 | 0.598 |
| MMQA | MGRAG | **0.853** | **0.744** | **0.676** | **0.722** |
| Arxiv | O.IE | 0.618 | 0.526 | 0.177 | 0.138 |
| Arxiv | STAT | 0.640 | 0.576 | 0.184 | 0.145 |
| Arxiv | VF2 | 0.871 | 0.650 | 0.270 | 0.239 |

Table 6.3 (continued)

| Dataset | Method | SIM. | ACC. | ROG. | BLEU |
|---|---|---|---|---|---|
| Arxiv | ISMAGS | 0.868 | 0.643 | 0.266 | 0.231 |
| Arxiv | SM | 0.866 | 0.670 | 0.263 | 0.244 |
| Arxiv | VJ | 0.872 | 0.667 | 0.269 | 0.245 |
| Arxiv | RRWM | 0.874 | 0.675 | 0.257 | 0.243 |
| Arxiv | IPFP | 0.814 | 0.581 | 0.212 | 0.203 |
| Arxiv | BGE | 0.874 | 0.668 | 0.270 | 0.244 |
| Arxiv | SLOT | 0.872 | 0.653 | 0.269 | 0.233 |
| Arxiv | FGW | 0.868 | 0.651 | 0.264 | 0.230 |
| Arxiv | MGRAG | **0.876** | **0.680** | **0.273** | **0.250** |
| Manual | O.IE | 0.520 | 0.459 | 0.245 | 0.209 |
| Manual | STAT | 0.509 | 0.443 | 0.234 | 0.182 |
| Manual | VF2 | 0.817 | 0.612 | 0.377 | 0.306 |
| Manual | ISMAGS | 0.817 | 0.612 | 0.382 | 0.307 |
| Manual | SM | 0.813 | 0.614 | 0.378 | 0.303 |
| Manual | VJ | 0.808 | 0.608 | 0.372 | 0.298 |
| Manual | RRWM | 0.811 | 0.614 | 0.377 | 0.303 |
| Manual | IPFP | 0.800 | 0.594 | 0.369 | 0.289 |
| Manual | BGE | 0.815 | 0.615 | 0.375 | 0.303 |
| Manual | SLOT | 0.813 | 0.611 | 0.375 | 0.301 |
| Manual | FGW | 0.820 | 0.600 | 0.374 | 0.292 |
| Manual | MGRAG | **0.822** | **0.617** | **0.384** | **0.309** |
| Web | O.IE | 0.466 | 0.290 | 0.052 | 0.020 |
| Web | STAT | 0.485 | 0.298 | 0.101 | 0.095 |
| Web | VF2 | 0.677 | 0.507 | 0.156 | 0.088 |
| Web | ISMAGS | 0.675 | 0.509 | 0.155 | 0.079 |
| Web | SM | 0.676 | 0.507 | 0.156 | 0.056 |
| Web | VJ | 0.679 | 0.508 | 0.160 | 0.056 |
| Web | RRWM | 0.675 | 0.503 | 0.157 | 0.066 |
| Web | IPFP | 0.666 | 0.493 | 0.149 | 0.055 |
| Web | BGE | 0.687 | 0.510 | 0.156 | 0.095 |
| Web | SLOT | 0.676 | 0.498 | 0.154 | 0.116 |
| Web | FGW | 0.676 | 0.480 | 0.147 | 0.096 |

Table 6.3 (continued)

| Dataset | Method | SIM. | ACC. | ROG. | BLEU |
|---|---|---|---|---|---|
| Web | MGRAG | **0.701** | **0.512** | **0.166** | **0.146** |
| Wiki | O.IE | 0.504 | 0.498 | 0.230 | 0.200 |
| Wiki | STAT | 0.525 | 0.510 | 0.253 | 0.218 |
| Wiki | VF2 | 0.877 | 0.752 | 0.352 | 0.300 |
| Wiki | ISMAGS | 0.879 | 0.754 | 0.355 | 0.303 |
| Wiki | SM | 0.878 | 0.750 | 0.353 | 0.298 |
| Wiki | VJ | 0.873 | 0.751 | 0.349 | 0.297 |
| Wiki | RRWM | 0.878 | 0.743 | 0.355 | 0.301 |
| Wiki | IPFP | 0.869 | 0.735 | 0.348 | 0.296 |
| Wiki | BGE | 0.878 | 0.746 | 0.355 | 0.301 |
| Wiki | SLOT | 0.879 | 0.749 | 0.356 | 0.301 |
| Wiki | FGW | 0.878 | 0.747 | 0.352 | 0.300 |
| Wiki | MGRAG | **0.884** | **0.785** | **0.358** | **0.317** |
| Wit | O.IE | 0.492 | 0.465 | 0.233 | 0.210 |
| Wit | STAT | 0.489 | 0.453 | 0.205 | 0.198 |
| Wit | VF2 | 0.799 | 0.693 | 0.354 | 0.234 |
| Wit | ISMAGS | 0.797 | 0.694 | 0.355 | 0.231 |
| Wit | SM | 0.784 | 0.686 | 0.353 | 0.229 |
| Wit | VJ | 0.786 | 0.691 | 0.356 | 0.227 |
| Wit | RRWM | 0.795 | 0.679 | 0.347 | 0.230 |
| Wit | IPFP | 0.792 | 0.678 | 0.346 | 0.231 |
| Wit | BGE | 0.789 | 0.689 | 0.356 | 0.230 |
| Wit | SLOT | 0.793 | 0.693 | 0.351 | 0.227 |
| Wit | FGW | 0.794 | 0.691 | 0.349 | 0.223 |
| Wit | MGRAG | **0.803** | **0.697** | **0.359** | **0.238** |
| Recipe | O.IE | 0.551 | 0.425 | 0.211 | 0.202 |
| Recipe | STAT | 0.545 | 0.401 | 0.198 | 0.172 |
| Recipe | VF2 | 0.859 | 0.679 | 0.413 | 0.304 |
| Recipe | ISMAGS | 0.861 | 0.668 | 0.407 | 0.302 |
| Recipe | SM | 0.862 | 0.678 | 0.411 | 0.305 |
| Recipe | VJ | 0.857 | 0.672 | 0.413 | 0.295 |
| Recipe | RRWM | 0.852 | 0.670 | 0.406 | 0.298 |

Table 6.3 (continued)

| Dataset | Method | SIM. | ACC. | ROG. | BLEU |
|---|---|---|---|---|---|
| Recipe | IPFP | 0.847 | 0.661 | 0.395 | 0.221 |
| Recipe | BGE | 0.864 | 0.674 | 0.415 | 0.308 |
| Recipe | SLOT | 0.861 | 0.675 | 0.412 | 0.306 |
| Recipe | FGW | 0.860 | 0.677 | 0.409 | 0.305 |
| Recipe | MGRAG | **0.867** | **0.681** | **0.422** | **0.310** |

On MMQA, the full method reports accuracy 0.744, compared with 0.571 for OpenIE and 0.602 for STAT. These substitutions change the available extracted evidence. Their lower scores establish the importance of the evaluated construction choice for this setting. The evaluated extraction setting uses multimodal evidence and query-specific graphs.

Among MMQA matching alternatives, VF2 reaches accuracy 0.735, while MGRAG reaches 0.744. VF2 also has ROUGE-L of 0.673 compared with 0.676. On Web, BGE reaches accuracy 0.510 and MGRAG 0.512. On Wiki, ISMAGS reaches 0.754 and MGRAG 0.785. The margins are therefore uneven. The table supports a consistent advantage for the full matching choice across the reported metric rows, with the magnitude determined by dataset and metric.

The matching experiment measures downstream answers after each substitution. Its metrics combine the selected graph with the generation model's use of the context. A higher answer score indicates a more effective retrieval-and-answering composition under this protocol. The semantic matching mechanism itself is described by its entity, path, and threshold conditions in Section 6.5; the answer metrics provide empirical evidence for its use in MGRAG.

### 6.9.3 Queries requiring multiple hops

Table 6.4 reports results grouped by MRAMG query-hop count. For one-hop questions, VanillaRAG reports accuracy 0.613 and MGRAG 0.624. For six-hop questions, the reported values are 0.661 and 0.738. MGRAG also reports higher ROUGE-L and BLEU-1 for each hop group. The table's time entries favor VanillaRAG throughout: the one-hop values are 0.13 and 0.35 minutes, and the six-hop values are 0.18 and 0.43 minutes.

Table 6.4: Results by query hop count, reorganized from published Table 8, p. 2490 [1]. Time is in minutes, as reported. Each original row is shown as two method rows, with the query count repeated.

| Hops | Queries | Method | Time | ACC. | ROG. | BLEU |
|---|---|---|---|---|---|---|
| 1-hop | 201 | VanillaRAG | **0.13** | 0.613 | 0.379 | 0.240 |
| 1-hop | 201 | MGRAG | 0.35 | **0.624** | **0.425** | **0.286** |
| 2-hop | 1,237 | VanillaRAG | **0.10** | 0.588 | 0.304 | 0.193 |
| 2-hop | 1,237 | MGRAG | 0.37 | **0.601** | **0.335** | **0.230** |
| 3-hop | 2,357 | VanillaRAG | **0.16** | 0.633 | 0.329 | 0.238 |
| 3-hop | 2,357 | MGRAG | 0.41 | **0.659** | **0.355** | **0.260** |
| 4-hop | 499 | VanillaRAG | **0.19** | 0.697 | 0.338 | 0.259 |
| 4-hop | 499 | MGRAG | 0.44 | **0.712** | **0.373** | **0.293** |
| 5-hop | 286 | VanillaRAG | **0.18** | 0.678 | 0.319 | 0.205 |
| 5-hop | 286 | MGRAG | 0.45 | **0.733** | **0.392** | **0.282** |
| 6-hop | 220 | VanillaRAG | **0.18** | 0.661 | 0.305 | 0.176 |
| 6-hop | 220 | MGRAG | 0.43 | **0.738** | **0.377** | **0.249** |

The six-hop group contains 220 queries, whereas the three-hop group contains 2,357. These group sizes matter when interpreting the distribution of the evidence. The comparison shows a larger accuracy gap on the longest-hop group than on the shortest-hop group, alongside additional processing time. It supplies a direct account of the quality-cost trade-off for questions with different relational demands. The hop-specific values retain the measured trade-off for each question group.

## 6.10 Cache Quality and End-to-end Costs

Table 6.5 preserves the MMQA cache-system comparison. MGRAG reports accuracy 0.7442 and total time 4.3 hours. The vLLM and CacheBlend variants paired with VanillaRAG report accuracies 0.6593 and 0.6601, with total time 4.5 hours for both. Paired with HippoRAG2, they report accuracies 0.6335 and 0.6267, with total times of 8.8 and 8.3 hours. These are distinct retrieval-and-cache compositions, each identified in the table.

Table 6.5: Cache-system comparison on MMQA, reproduced from published Table 6, p. 2489 [1]. V. and H. identify VanillaRAG and HippoRAG2 retrieval pipelines. TTFT is in seconds and total time is in hours.

| System | SIM. | ACC. | ROG. | BLEU | TTFT | Time |
|---|---|---|---|---|---|---|
| vLLM (V.) | 0.8315 | 0.6593 | 0.6277 | 0.5867 | 3.2 s | 4.5 h |
| CacheBlend (V.) | 0.8315 | 0.6601 | 0.6417 | 0.5964 | **3.1 s** | 4.5 h |
| vLLM (H.) | 0.8381 | 0.6335 | 0.6659 | 0.5917 | 4.6 s | 8.8 h |
| CacheBlend (H.) | 0.8415 | 0.6267 | 0.6636 | 0.5917 | 4.4 s | 8.3 h |
| **MGRAG** | **0.8533** | **0.7442** | **0.6763** | **0.7215** | 3.2 s | **4.3 h** |

Time to first token (TTFT) gives a different view from total time. CacheBlend with VanillaRAG reports 3.1 seconds, while MGRAG and vLLM with VanillaRAG report 3.2 seconds. MGRAG combines the highest answer-quality values with a total time of 4.3 hours; CacheBlend with VanillaRAG has the shortest TTFT. The two latency measures should be read alongside their quality measurements, because a system can change one without changing the other in the same direction.

The inverse-ablation table also measures quality when graph-aware recomputation is added after path matching. Within the HippoRAG2 composition, accuracy changes from 0.7372 with path matching to 0.7442 with both components, while the total time changes from 4.1 to 4.3 hours. The additional computation is accompanied by changes in answer quality. Reporting both values preserves the contribution of the cache policy without assuming that every application of recomputation reduces total runtime. Its purpose is to allocate repeated computation to token states selected by the combined score.

The result also depends on the retrieval pipeline paired with the cache. Adding a cache policy to an existing system leaves that system's evidence representation in place. MGRAG's full result includes query-aware construction and path-based matching as well as graph-aware recomputation. The inverse ablations in Section 6.11 supply the closer comparison for the contribution of the graph feature after a retrieval choice has been fixed.

Table 6.6 reports time and token costs by module. On MMQA, graph construction accounts for 193.0 minutes, matching for 0.14 minutes, and answer generation for 8.9 minutes; the reported end-to-end time is 248.1 minutes. The construction and generation token entries are 3,085 and 1,431, with end-to-end token entry 4,516. The end-to-end measurement is retained separately from the listed module times. The original totals and their reported precision are preserved.

Table 6.6: Module costs reproduced from published Table 5, p. 2489 [1]. Time is in minutes. KGC, QA, and E2E cells retain the published time/token pairs for graph construction, answering, and end-to-end processing. Match denotes matching time. Token counts and E2E totals are preserved as reported.

| Dataset | Queries | KGC | Match | QA | E2E |
|---|---|---|---|---|---|
| MMQA | 951 | 193.0 / 3,085 | 0.14 | 8.9 / 1,431 | 248.1 / 4,516 |
| Arxiv | 200 | 61.9 / 3,946 | 0.23 | 16.2 / 2,944 | 83.9 / 6,890 |
| Manual | 390 | 76.1 / 4,285 | 0.04 | 23.9 / 2,556 | 109.7 / 6,841 |
| Web | 750 | 143.8 / 4,623 | 0.27 | 42.9 / 3,101 | 196.5 / 7,723 |
| Wiki | 500 | 164.2 / 6,629 | 0.30 | 52.1 / 6,267 | 241.5 / 12,896 |
| Wit | 600 | 140.6 / 7,660 | 0.21 | 42.6 / 6,539 | 196.5 / 14,199 |
| Recipe | 2,360 | 525.7 / 4,351 | 1.74 | 191.3 / 3,173 | 808.7 / 7,525 |

Graph construction is the largest listed time component on all seven datasets. Recipe reports 525.7 minutes for construction, 1.74 for matching, and 191.3 for answering, with end-to-end time 808.7 minutes. On the smaller Arxiv query set, the corresponding values are 61.9, 0.23, and 16.2 minutes, with end-to-end time 83.9. These measurements locate repeated extraction calls as a substantial cost within the evaluated pipeline, even though graph matching has an exponential worst-case bound.

The distinction between computational bounds and measured module costs is useful here. The graphs are built from query-selected evidence, so the matcher processes small instances in these experiments. Construction invokes the multimodal model on the selected chunks. Reducing the number of extracted chunks or the cost of each extraction could therefore affect the observed runtime substantially. Such changes are prospective design directions; the table records the published pipeline's cost.

## 6.11 Ablations and Parameter Sensitivity

### 6.11.1 Component ablations

The MMQA ablations in Table 6.7 change matching, coarse encoding, and pipeline composition. Removing graph matching constructs the query-specific graph from a fixed number of chunks and reports accuracy 0.7260, compared with 0.7442 for MGRAG. Replacing caption-based coarse ranking with direct CLIP encoding [24] reports accuracy 0.7376. The Vanilla MGRAG variant applies path matching and graph-aware caching within the VanillaRAG pipeline

with $k = 5$; it reports accuracy 0.6887.

Table 6.7: MMQA component ablations, reproduced from published Table 9, p. 2490 [1]. Time is in hours.

| Model | SIM. | ACC. | ROG. | BLEU | Time |
|---|---|---|---|---|---|
| Without graph matching | 0.8362 | 0.7260 | 0.6657 | 0.5774 | 4.4 h |
| With CLIP encoder | 0.8513 | 0.7376 | 0.6744 | 0.7008 | 4.5 h |
| Vanilla MGRAG, $k = 5$ | 0.8260 | 0.6887 | 0.6266 | 0.5773 | 4.7 h |
| **MGRAG** | **0.8533** | **0.7442** | **0.6763** | **0.7215** | **4.3 h** |

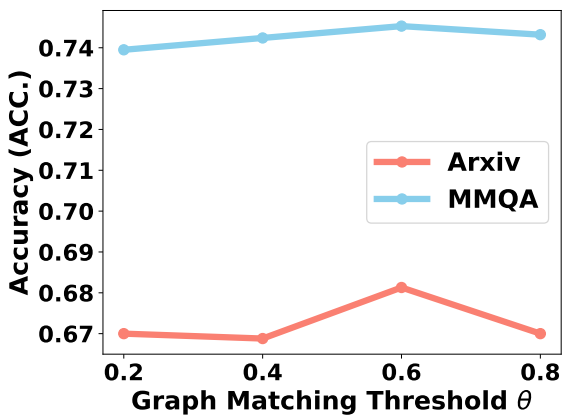
The full system also has the largest SIM., ROUGE-L, and BLEU-1 entries in this table. Its reported time is 4.3 hours, compared with 4.4 hours without graph matching, 4.5 hours with CLIP, and 4.7 hours for Vanilla MGRAG. The comparison supports the particular coarse-to-fine composition used in the published system. It also identifies the scope of the encoder observation: caption-based ranking performs better than the tested CLIP substitution on MMQA with this pipeline and backbone.

### 6.11.2 Adding components to other pipelines

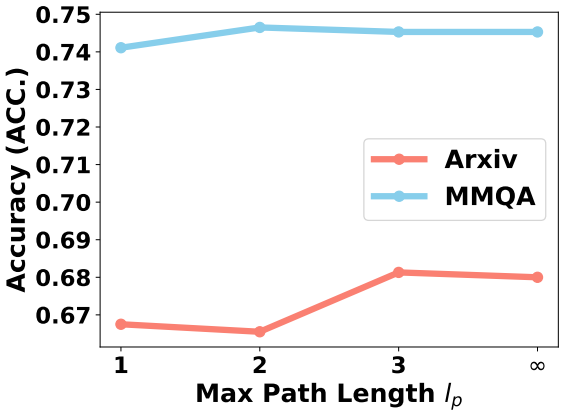
Table 6.8 retains the inverse ablations. Adding graph-aware KV recomputation to HippoRAG2 changes accuracy from 0.6335 to 0.6855 and time from 7.5 to 7.6 hours. Adding path-based semantic matching gives accuracy 0.7372 and time 4.1 hours. Combining both gives accuracy 0.7442 and time 4.3 hours. The latter configuration shares MGRAG's reported values because the publication identifies its retrieval and generation setting with the full method once both components are incorporated.

Table 6.8: MMQA inverse ablations, reproduced from published Table 10, p. 2491 [1]. PSGM and GKV denote path-based semantic graph matching and graph-aware KV recomputation. Time is in hours.

| Model | SIM. | ACC. | ROG. | BLEU | Time |
|---|---|---|---|---|---|
| Hippo. | 0.8381 | 0.6335 | 0.6659 | 0.5917 | 7.5 h |
| Hippo.+GKV | 0.8392 | 0.6855 | 0.6651 | 0.7013 | 7.6 h |
| Hippo.+PSGM | 0.8416 | 0.7372 | 0.6720 | 0.7128 | 4.1 h |
| Hippo.+PSGM+GKV | **0.8533** | **0.7442** | **0.6763** | **0.7215** | 4.3 h |
| Vanilla. | 0.8362 | 0.7053 | 0.5942 | 0.6645 | **2.6 h** |




(a) Similarity threshold $\theta$.




(b) Maximum path length $l_p$.

Figure 6.5: MGRAG matching-parameter sensitivity. Original subfigures of Figure 5, p. 2491 [1].

Table 6.8 (continued)

| Model | SIM. | ACC. | ROG. | BLEU | Time |
|---|---|---|---|---|---|
| Vanilla.+PSGM | 0.8460 | 0.7175 | 0.6466 | 0.6973 | 4.7 h |
| Vanilla.+PSGM+GKV | 0.8485 | 0.7232 | 0.6563 | 0.7001 | 4.8 h |
| **MGRAG** | **0.8533** | **0.7442** | **0.6763** | **0.7215** | 4.3 h |

For VanillaRAG, the base accuracy is 0.7053 at 2.6 hours. Adding path matching gives 0.7175 at 4.7 hours, and adding graph-aware recomputation as well gives 0.7232 at 4.8 hours. Thus, the component additions increase quality in these measured compositions, while their time changes depend on the host pipeline. The HippoRAG2 and VanillaRAG results make that dependence visible. They also distinguish the value of a component from the full system's overall comparison.

### 6.11.3 Matching threshold and path length

Figure 6.5 reproduces the published parameter curves. The experiments vary the semantic threshold $\theta$ and maximum path length $l_p$, including lengths 1, 2, 3, and unrestricted length. The reported discussion identifies $\theta = 0.6$ and $l_p = 3$ as effective choices in these tests. The curves are retained as figures; no numerical points are estimated from their plotted pixels.

The threshold changes which entity correspondences and path extensions can pass validation. A higher value demands closer embedding agreement, while a lower value permits more candidates. The path-length cap determines how many relations may be used to express a query

relation. Its effect therefore concerns both representational flexibility and search. These roles explain why the two parameters belong to the matching mechanism, while their empirical choice remains tied to the reported evaluation.

## 6.12 Discussion and Chapter Summary

MGRAG gives structure three connected responsibilities. Query-aware extraction determines which facts enter the graph. Path-based matching determines whether those facts express the relational requirements of the query. Graph features then influence the selection of token states for recomputation. The graph is consequently involved in constructing, selecting, and processing evidence, rather than serving only as a store from which a fixed number of items is retrieved.

The experiments support this design at several levels. Complete-system results report the highest values across the seven datasets' four answer metrics. Construction and matching substitutions evaluate the evidence representation and correspondence choices. The hop-group comparison shows additional quality gains for questions requiring more relational steps, together with added time. Cache comparisons and inverse ablations retain the context-serving contribution and expose its dependence on the surrounding retrieval pipeline.

Several conditions delimit the interpretation. Captions influence the coarse order of image evidence, and model-generated extraction determines which facts are available for matching. A valid semantic match is assessed against the extracted query graph and an embedding threshold; answer correctness is evaluated downstream. The maximum construction and path budgets restrict the search. The published runtime measurements use one hardware and model configuration, and the tables do not provide the fixed numerical recomputation budget needed to attribute a cost to a particular $b$ value. These conditions identify the representation and computation choices on which the reported results depend.

For the thesis's central question, the chapter establishes a concrete extension from connected textual evidence to query-dependent multimodal evidence. Semantic paths allow relations to be represented at different granularities across the query and evidence graphs. Selective KV recomputation shows a further use of the retrieved structure in context processing. The following discussion compares these roles with the representations and retrieval objects studied in KGLink, GORAG, and AGRAG, using their respective tasks and published evidence.

# BIBLIOGRAPHY


[1] Y. Wang *et al.*, "MGRAG: Semantic subgraph matching and graph-aware caching for multimodal retrieval-augmented generation," *Proceedings of the VLDB Endowment*, vol. 19, no. 9, pp. 2480–2493, 2026. DOI: 10.14778/3819518.3819565 (cit. on pp. 1–3, 5, 12, 14, 16, 18, 22–26).

[2] S. Xiao *et al.*, *C-Pack: Packaged resources to advance general chinese embedding*, 2023 (cit. on pp. 4, 18).

[3] L. P. Cordella *et al.*, "A (sub) graph isomorphism algorithm for matching large graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 26, no. 10, pp. 1367–1372, 2004 (cit. on pp. 9, 18).

[4] Y. Wang *et al.*, *MGRAG: Semantic subgraph matching and graph-aware caching for multimodal retrieval-augmented generation [technical report]*. [Online]. Available: https://github.com/Wyb0627/MGRAG/blob/main/tech_report.pdf (cit. on p. 11).

[5] W. Kwon *et al.*, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626 (cit. on p. 14).

[6] J. Yao *et al.*, "Cacheblend: Fast large language model serving for rag with cached knowledge fusion," in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025, pp. 94–109 (cit. on p. 14).

[7] A. Talmor *et al.*, "Multimodalqa: Complex question answering over text, tables and images," *International Conference on Learning Representations*, 2021 (cit. on p. 14).

[8] Q. Yu *et al.*, "Mramg-bench: A comprehensive benchmark for advancing multimodal retrieval-augmented multimodal generation," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2025, pp. 3616–3626 (cit. on p. 14).

[9] S. Bai *et al.*, "Qwen2.5-VL technical report," *arXiv preprint arXiv:2502.13923*, 2025 (cit. on p. 15).

[10] G. Izacard *et al.*, "Unsupervised dense information retrieval with contrastive learning," *Transactions on Machine Learning Research*, (cit. on p. 15).

[11] M. Douze *et al.*, "The faiss library," 2024 (cit. on p. 15).

[12] "nano-graphrag: A simple, easy-to-hack GraphRAG implementation." (), [Online]. Available: https://github.com/gusye1234/nano-graphrag (cit. on p. 15).

[13] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, Barcelona, Spain: Association for Computational Linguistics, 2004, pp. 74–81 (cit. on p. 15).

[14] K. Papineni *et al.*, "Bleu: A method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318 (cit. on p. 15).

[15] S. Es *et al.*, "Ragas: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158 (cit. on p. 15).

[16] G. Angeli *et al.*, "Leveraging linguistic structure for open domain information extraction," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2015, pp. 344–354 (cit. on p. 18).

[17] M. Houbraken *et al.*, "The index-based subgraph matching algorithm with general symmetries (ismags): Exploiting symmetry for faster subgraph enumeration," *PloS one*, vol. 9, no. 5, e97896, 2014 (cit. on p. 18).

[18] M. Leordeanu *et al.*, "A spectral technique for correspondence problems using pairwise constraints," in *Tenth IEEE International Conference on Computer Vision (ICCV'05) Volume 1*, IEEE, vol. 2, 2005, pp. 1482–1489 (cit. on p. 18).

[19] S. Fankhauser *et al.*, "Speeding up graph edit distance computation through fast bipartite matching," in *International Workshop on Graph-Based Representations in Pattern Recognition*, Springer, 2011, pp. 102–111 (cit. on p. 18).

[20] M. Cho *et al.*, "Reweighted random walks for graph matching," in *European conference on Computer vision*, Springer, 2010, pp. 492–505 (cit. on p. 18).

[21] M. Leordeanu *et al.*, "An integer projected fixed point method for graph matching and map inference," *Advances in neural information processing systems*, vol. 22, 2009 (cit. on p. 18).

[22] J. Tang *et al.*, "Robust attributed graph alignment via joint structure learning and optimal transport," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, IEEE, 2023, pp. 1638–1651 (cit. on p. 18).

[23] J. Tang *et al.*, "Fused gromov-wasserstein alignment for graph edit distance computation and beyond," *Proceedings of the VLDB Endowment*, vol. 18, no. 10, pp. 3641–3654, 2025 (cit. on p. 18).

[24] A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*, PmLR, 2021, pp. 8748–8763 (cit. on p. 24).